

InsightSwarm — The Math, the Algorithms, and How It Thinks

Explained Simply — No Engineering Degree Required

Last updated: April 2026

This file covers the actual formulas and algorithms taken directly from the InsightSwarm codebase. Every number and formula shown here is **real** — not made up for this guide.

Table of Contents

1. What Is a Number Score? (The Foundation)
2. How the System Checks If It Already Knows the Answer — Semantic Cache
3. How It Decides How Hard a Claim Is — Complexity Estimation
4. How It Scores Website Trustworthiness — Trust Scorer
5. How It Checks Sources — The Verification Algorithm
6. The Final Score Formula — The Moderator's Math
7. How It Detects Fake Arguments — Fallacy Detection
8. How It Fixes Its Own Overconfidence Problem — Calibration
9. How It Spots Contradictions Between Sources
10. How It Explains Its Own Decisions — The XAI Engine
11. How the AI Providers Are Managed — Circuit Breaker
12. Putting It All Together — The Full Picture

Section 1 : What Is a Number Score? (The Foundation)

Before diving in, one simple idea to understand everything else:

Almost everything in InsightSwarm is turned into a number between 0.0 and 1.0.

- 0.0 means “completely no” or “zero trust” or “completely false”
- 1.0 means “completely yes” or “full trust” or “completely true”
- 0.5 means “right in the middle — not sure either way”

So when you see a number like 0.82, think: *“82% of the way from no to yes.”*
When you see 0.3, think: *“only 30% — quite weak.”*

This is used for: - How similar two sentences are (similarity score) - How trustworthy a website is (trust score) - How confident the system is in its answer (confidence score) - How good an argument is (quality score)

Every formula in this file is just a way of combining several of these 0-to-1 numbers into one final 0-to-1 number.

Section 2 : How the System Checks If It Already Knows the Answer — The Semantic Cache

The Problem

Running the full debate takes 35 to 60 seconds. If someone already checked “*Do vaccines cause autism?*” yesterday, there is no need to run the whole thing again when someone else asks “*Can vaccines lead to autism?*” — they mean the same thing.

But you cannot just check if the words are identical. The two sentences above use completely different words but mean the same thing. So the system needs to understand **MEANING**, not just words.

How Meaning Is Turned Into Numbers

InsightSwarm uses a pre-trained AI model called **all-MiniLM-L6-v2** to turn every claim into a list of **384 numbers**. This list is called an **embedding**.

Think of it like GPS coordinates. Every place in the world can be described with just two numbers (latitude and longitude). Similarly, every sentence can be described with 384 numbers. Sentences that **mean** the same thing will have very similar sets of 384 numbers — just like how two buildings on the same street have similar GPS coordinates.

Example: - “Do vaccines cause autism?”

Might become: [0.12, -0.34, 0.87, 0.02, ...] (384 numbers) - “Can vaccines lead to autism?”

Might become: [0.13, -0.32, 0.85, 0.03, ...] (384 numbers)

These are very close. The first sentence about “*Earth orbits the Sun*” would produce completely different numbers, far away from the vaccine ones.

The Math: Cosine Similarity

Once we have two lists of 384 numbers, we measure how “pointed in the same direction” they are. This is called **cosine similarity**.

Imagine two arrows drawn from the same starting point. If both arrows point in exactly the same direction, they are identical (similarity = 1.0). If they point in opposite directions, similarity = -1.0. If they are at a right angle to each other, similarity = 0.

The formula is:

$$\text{similarity} = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \times \|\mathbf{B}\|)$$

Where “ $\cdot$ ” means dot product: multiply each pair of matching numbers, then sum.

In the actual code (`src/orchestration/cache.py`):

- `query_norm` = `query_embedding` / length of `query_embedding`
- `embeddings_norm` = all stored embeddings / length of each one
- `similarities` = dot product of (`embeddings_norm` with `query_norm`)
- `best_similarity` = the highest similarity found

The Threshold

- If `best_similarity` $\geq$ 0.85 $\rightarrow$ return the cached answer immediately.
- If `best_similarity` < 0.85 $\rightarrow$ run the full debate.

Why 0.85 and not 1.0?

0.85 means “at least 85% similar in meaning.” Asking for 1.0 would only match exact identical sentences. 0.85 catches paraphrases that mean the same thing.

Performance: finding the cached answer takes < **1 second**.

Without the cache: **35–60 seconds**.

The code also builds a matrix of all stored embeddings once and caches it, so comparisons happen simultaneously — much faster than one-by-one.

Section 3 : How It Decides How Hard a Claim Is — Complexity Estimation

The Problem

- “*The Earth is round*” takes 2 debate rounds to confirm.
- “*Will AI replace all software developers by 2030?*” needs 4 rounds and 7 sources.

Running the same depth of analysis for both wastes time or under-analyses.

The Solution

Before the debate starts, InsightSwarm scores the claim on **four dimensions**:

Dimension 1: Semantic Complexity (how many ideas are packed in?)

- Counts words (longer = more complex)
- Counts capitalised words not at the start (likely named entities)
- Counts commas and the words “and”/“or” (indicate multiple parts)

Formula: $\text{Semantic} = (0.4 \times \text{length_score}) + (0.3 \times \text{entity_score}) + (0.3 \times \text{clause_score})$

Where: $\text{length_score} = \min(1.0, \text{number_of_words} / 30)$

$\text{entity_score} = \min(1.0, \text{number_of_capital_words} / 5)$

$\text{clause_score} = \min(1.0, \text{number_of_commas_and_conjunctions} / 3)$

Dimension 2: Domain Complexity (specialised topic?) The system has lists of technical vocabulary for four domains: - **Medical:** efficacy, antibody, pathogen, clinical... - **Scientific:** hypothesis, correlation, statistical... - **Legal:** statute, precedent, jurisdiction... - **Economic:** gdp, inflation, monetary, fiscal...

If a medical word is found $\rightarrow$ base difficulty 0.9

If scientific $\rightarrow$ 0.8, legal $\rightarrow$ 0.7, economic $\rightarrow$ 0.6, none $\rightarrow$ 0.2

Formula: $\text{Domain} = \text{base_difficulty} \times (0.6 + 0.4 \times \text{term_density})$

$\text{term_density} = \min(1.0, \text{count_of_found_terms} / 3)$

Dimension 3: Temporal Complexity (time involved?)

- “will”, “by 2030”, “going to” $\rightarrow$ 0.9 (future prediction, hardest to verify)
- “historically”, “in the past” $\rightarrow$ 0.7

- Specific year like “2019” $\rightarrow$ 0.6
- Otherwise $\rightarrow$ 0.3

Dimension 4: Evidence Availability (is there a lot written about this?)

- Common topics (covid, vaccine, climate...) $\rightarrow$ 0.9 (lots of evidence)
- Mentions 2+ named people/places $\rightarrow$ 0.6
- Abstract claims (philosophy, happiness...) $\rightarrow$ 0.2
- Default $\rightarrow$ 0.5

Combining All Four Dimensions

$$\text{Overall complexity} = (0.3 \times \text{Semantic}) + (0.3 \times \text{Domain}) + (0.2 \times \text{Temporal}) + (0.2 \times (1.0 - \text{Evidence_Availability}))$$

Notice that Evidence is flipped: low availability = high complexity.

Result Table

Complexity score	Label	Debate Rounds	Min Sources Required
0.7	very high	4	7
0.5 – 0.7	high	3	5
0.3 – 0.5	medium	3	4
< 0.3	low	2	3

Also: if complexity $\geq$ 0.7 **or** the claim is medical/legal, the system flags it as “*requires expert review*”, triggering the Human-in-the-Loop alert sooner.

Section 4 : How It Scores Website Trustworthiness — Trust Scorer

Not all websites are equally reliable. InsightSwarm assigns every URL a **trust score**

between 0.0 and 1.0 based on the domain name.

Code location: `src/utils/trust_scorer.py`

Score Table (taken directly from the code)

Score	Tier	Examples
1.0	AUTHORITATIVE	.gov, .edu, who.int, cdc.gov, nasa.gov, nih.gov, nature.com, science.org, thelancet.com
0.8	CREDIBLE	reuters.com, apnews.com, nytimes.com, wsj.com, bbc.co.uk, theguardian.com, economist.com, bloomberg.com
0.5	GENERAL	Any site not in the above categories (default)
0.3	UNDIRECTED	twitter.com, x.com, facebook.com, reddit.com, medium.com, substack.com
0.1	UNRELIABLE	dailymail.co.uk, infowars.com, naturalnews.com, thegatewaypundit.com, breitbart.com

Usage: The trust score for each verified source feeds directly into the final verdict formula (see Section 6). A claim backed by **nature.com** (1.0) has far more weight than one backed by a random blog (0.5) or a Reddit post (0.3).

Section 5 : How It Checks Sources — The Verification Algorithm

After the debate, every URL mentioned by either agent goes through this process.

Step 1 – Filter Invalid URLs

- If the URL is not a real web address format → `INVALID_URL`, skip.
- If the URL points to a private/internal network (192.168.x.x, localhost...) → block immediately (SSRF protection).

Step 2 – Fetch the Page

- Send an HTTP request, wait **10 seconds** max.
- Site times out → `TIMEOUT`
- Page returns 404 → `NOT_FOUND` (the URL was likely made up)
- Page is behind a login/paywall → `PAYWALL_RESTRICTED`

Step 3 – Extract the Text

- Use `BeautifulSoup` to strip HTML and extract readable text.
- Crop to the most relevant **500–2,000 characters**.

Step 4 – Measure How Much the Page Supports the Claim

- Encode both the claim and the page text into **384-number embeddings** (same method as the cache).
- Calculate **cosine similarity** between them.

Threshold:

- Similarity **0.82** → `VERIFIED`
- Below 0.82 → `CONTENT_MISMATCH`

Why 0.82 here but 0.85 for the cache?

The cache asks “is this the same question?” — needs higher precision.

Verification asks “does this page support the claim?” — a bit more lenient; a page can discuss a topic without repeating the exact claim.

Step 5 – Assign Trust Score

- Look up the domain in the Trust Scorer (Section 4).
- Apply the score to the result.

Step 6 – Final Label

Label	Meaning
VERIFIED	Page loaded, content matched (0.82), trust applied
NOT_FOUND	404 or DNS error — URL probably made up
CONTENT_MISMATCH	Real page, but content does not support the claim
TIMEOUT	No response within 10 seconds
PAYWALL_RESTRICTED	Behind a login wall
INVALID_URL	Not a real URL format

Speed: All URLs are checked in parallel — **5 workers** at the same time. So checking 10 URLs takes roughly as long as checking 2 one-by-one.

Section 6 : The Final Score Formula — The Moderator’s Math

After the debate and verification, the Moderator computes a single confidence number. **This is the most important formula in the whole system.**

Taken directly from `src/agents/moderator.py`

Composite confidence =
 $(\text{argument_quality} \times 0.30) +$
 $(\text{avg_verification} \times 0.30) +$
 $(\text{avg_trust} \times 0.20) +$
 $(\text{consensus_score} \times 0.20)$

Where Each Component Comes From

Argument Quality (weight 30%)

The AI Moderator reads all arguments and gives them a quality score (0.0–1.0). Strong logic with good citations scores high; fallacies make it low (see Section 7).

Average Verification Rate (weight 30%)

For each side (PRO and CON), a *trust-weighted* verification rate is calculated:

For each source: $\text{weight} = \text{trust_score} \times \text{confidence_of_match}$

For each agent:

$\text{weighted_verification} = (\text{sum of weights for VERIFIED sources}) /$
 $(\text{sum of weights for all sources})$

Then:

$\text{avg_verification} = (\text{PRO_rate} + \text{CON_rate}) / 2$

This rewards sides with real, high-quality sources and penalises fake or mismatched ones.

Average Trust (weight 20%)

Average of trust scores for all verified sources.

If no sources were verified → defaults to 0.5.

Consensus Score (weight 20%)

Before the debate, the system checks if the claim has settled scientific consensus.

- Consensus matches Moderator’s verdict → 0.8
- Consensus says “DEBATE” (no consensus) → 0.5
- They disagree → 0.2

Final Clamping:

$\text{composite} = \max(0.0, \min(1.0, \text{composite}))$

Example Walkthrough

Claim: “*Coffee reduces type 2 diabetes risk*”

- Argument quality = 0.72
 - Avg verification = 0.65
 - Avg trust = 0.70 (mostly news and journal sources)
 - Consensus = 0.50 (no settled consensus)
- $$\begin{aligned}\text{composite} &= (0.72 \times 0.30) + (0.65 \times 0.30) + (0.70 \times 0.20) + \\ &\quad (0.50 \times 0.20) \\ &= 0.216 + 0.195 + 0.140 + 0.100 \\ &= \mathbf{0.651} \rightarrow \text{PARTIALLY TRUE, 65\% confidence}\end{aligned}$$

Fallback Behaviour:

- All providers out of requests $\rightarrow$ RATE_LIMITED, confidence 0.0
- Technical crash $\rightarrow$ SYSTEM_ERROR, confidence 0.0

Section 7 : How It Detects Fake Arguments — Fallacy Detection

Code: src/novelty/argumentation_analysis.py

The Argumentation Analyzer uses **pattern matching** to find **10 types** of logical fallacies.

How Pattern Matching Works

Each fallacy has a list of text patterns (regular expressions). The system scans every argument for these patterns.

Simplified examples:

- **Ad Hominem** (attacking the person): words like “idiots”, “morons”, “can’t trust them”
- **Slippery Slope** (chain reaction without proof): phrases like “if X then eventually...”, “leads to ... which leads to”
- **Cherry Picking** (only convenient facts): “one study found”, “conveniently ignores”

Penalty:

$\text{fallacy_penalty} = \text{number_of_fallacies_found} \times 0.15$

Argument Quality Score Formula

For each argument:

Step 1 – Structure Score

- $\text{length_score} = 1.0$ if length is 50–1000 characters
- $\text{structure_score} = \text{variety of sentence lengths (more variety = better)}$
- $\text{evidence_score} = \text{fraction of cited URLs actually mentioned in the text}$

$\text{structure} = (0.3 \times \text{length_score}) + (0.3 \times \text{structure_score}) + (0.4 \times \text{evidence_score})$

Step 2 – Bonuses & Penalties

- $\text{citation_bonus} = +0.20$ if source-to-claim ratio is between 0.5 and 2.0
- $\text{fallacy_penalty} = -(\text{number_of_fallacies} \times 0.15)$
- $\text{rhetoric_penalty} = -(\text{rhetoric_score} \times 0.05)$

(rhetoric score measures superlatives, certainty claims, emotional language)

$\text{Final quality} = \max(0.0, \min(1.0, \text{structure} + \text{citation_bonus} - \text{fallacy_penalty} - \text{rhetoric_penalty}))$

Quality Classification

- 0.8 → excellent

- $0.6 \rightarrow \text{good}$
- $0.4 \rightarrow \text{fair}$
- $< 0.4 \rightarrow \text{poor}$

Rhetoric Score (argumentative tricks)

- Repetition $\times 0.1$
- Rhetorical questions $\times 0.3$
- Superlatives $\times 0.2$
- Certainty claims $\times 0.2$
- (Certainty – hedging) $\times 0.2$
(hedging words: “maybe”, “perhaps”, “might” – show uncertainty)

Debate Comparison

After analysing all rounds: - **pro_average** = average of PRO quality scores
 - **con_average** = average of CON quality scores
 - **quality_gap** = $|\text{pro_average} - \text{con_average}|$
 - **higher_quality_side** = whichever is higher
 - **debate_quality** = “high” if both sides > 0.6 , else “low”

Section 8 : How It Fixes Its Own Overconfidence Problem — Calibration

Code: *src/novelty/confidence_calibration.py*

The Problem: AI Models Are Too Modest

AI models often give conservative scores (e.g., 0.55) when evidence is strong. This is **under-confidence bias**.

Example:

Claim: “*Water boils at 100°C at sea level.*”

- Raw Moderator verdict: TRUE, 0.58 confidence (too cautious)
- After calibration: TRUE, 0.78 confidence (more honest)

How Under-Confidence Is Detected

The calibrator checks for **all** of these conditions: 1. Verdict is TRUE or FALSE (not ambiguous) 2. Raw confidence < 0.6 3. Source quality > 0.7 (evidence is actually good) 4. Debate asymmetry > 0.5 (one side clearly dominated)

Or **any** of these alone: - Source quality > 0.8 AND confidence < 0.7 - Debate asymmetry > 0.7 AND confidence < 0.65

Source Quality: The Geometric Mean

InsightSwarm uses the **geometric mean** (not the regular arithmetic mean) to punish weak sources more heavily.

Regular mean example:

Scores: 0.9, 0.9, 0.1 $\rightarrow (0.9+0.9+0.1)/3 = 0.63$ (looks okay)

Geometric mean example:

$(0.9 \times 0.9 \times 0.1) = 0.081 = 0.43$ (properly punished for one weak source)

Formula:

`geometric_mean = e^(mean of logarithms)`

In code: `np.exp(np.mean(np.log([max(t, 0.01) for t in trust_scores])))`

The *max(t, 0.01)* prevents crash if any score is exactly 0.

Debate Asymmetry

Measures how lopsided the debate was:

$$\begin{aligned}\text{argument_length_asymmetry} &= \frac{|\text{PRO_chars} - \text{CON_chars}|}{(\text{PRO_chars} + \text{CON_chars})} \\ \text{source_count_asymmetry} &= \frac{|\text{PRO_sources} - \text{CON_sources}|}{\dots}\end{aligned}$$

$$\begin{aligned} & (\text{PRO_sources} + \text{CON_sources}) \\ \text{asymmetry} &= (0.6 \times \text{arg_asym}) + (0.4 \times \text{src_asym}) \end{aligned}$$

Range: 0 (perfectly equal) to 1 (completely one-sided).

The Boost Formula

When under-confidence is detected:

$$\text{evidence_strength} = (0.6 \times \text{source_quality}) + (0.4 \times \text{debate_asymmetry})$$

Only boost if $\text{evidence_strength} > 0.6$.

$$\text{boost} = \min(0.25, (\text{evidence_strength} - 0.60) \times 0.50)$$

$$\text{calibrated} = \text{raw_confidence} + \text{boost} \times (1 - \text{raw_confidence})$$

The second part ensures diminishing returns. Final cap: **0.95** (never 100% certain).

Example: - raw_confidence = 0.55

- source_quality = 0.82

- debate_asymmetry = 0.65

- evidence_strength = 0.752

- boost = 0.076

- calibrated = $0.55 + 0.076 \times 0.45 = \mathbf{0.584}$ (a modest, honest upward adjustment)

Expected Calibration Error (ECE)

ECE measures how well confidence matches accuracy.

For 100 claims at 70% confidence, ideally 70 should be correct.

$$\text{ECE} = \text{average of } |\text{predicted_confidence} - \text{actual_accuracy}| \text{ across bins}$$

InsightSwarm reduces ECE from **0.31** (single AI) to **0.24**.

Lower is better — our confidence scores are more honest.

Section 9 : How It Spots Contradictions Between Sources

Code: `src/novelty/contradiction_detection.py`

Two sources can both be real and verified, but still say opposite things.

Example:

- Source A (2014): *“High-fat diets are harmful”*
- Source B (2023): *“Certain high-fat diets are beneficial”*

InsightSwarm detects three types:

Type 1: Directional Contradiction

Counts positive/negative signal words:

Positive words: beneficial, effective, improves, reduces risk...

Negative words: harmful, ineffective, worsens, increases risk...

Contradiction detected when: - Source 1 is positive (positives > negatives, positives - 2)

- AND Source 2 is negative (negatives > positives, negatives - 2)

Type 2: Temporal Contradiction

Extracts all years mentioned. If the **most recent year** in two sources differs by 5 years:

- Gap 10 years → severity “high” (science may have changed significantly)
- Gap 5 years → severity “medium”

Type 3: Jurisdictional Contradiction

Checks if sources belong to different regulatory bodies: - US: “fda”, “.gov”

- EU: “ema”, “.eu”

- UK: “mhra”, “.uk”

- WHO: “who”, “world health”

A drug approved by the FDA might not be approved by the EMA — hence, legitimate contradiction.

Resolution Strategies

- `temporal_priority_newer` – if gap 10 years, trust the newer source
- `evolving_science` – if gap 5–10 years, science evolved

- `context_dependent` – if jurisdictional, answer depends on location
- `quality_weighted` – otherwise, use trust scores to decide

Verdict Modifiers

- 2 temporal contradictions → `EVOLVING_SCIENCE` modifier
- 1 jurisdictional contradiction → `CONTEXT_DEPENDENT` modifier
- Otherwise → `PARTIALLY_TRUE`

Only **VERIFIED** sources are compared.

Section 10 : How It Explains Its Own Decisions — The XAI Engine

Code: `src/novelty/explainability.py`

XAI = Explainable AI. If the system gives an answer, it must also explain **why**.

Feature Importance (What Mattered Most?)

Calculates how much each factor contributed to the final verdict:

- `source_trust_contribution = trust_asymmetry × 0.30`
- `verification_contribution = verification_asymmetry × 0.30`
- `argument_quality_contribution = argument_quality_score × 0.20`
- `consensus_contribution = consensus_score × 0.20`

These are **normalised** to sum to 1.0.

Example output: - source_trust: 0.42 (most important)

- verification_rate: 0.28
- argument_quality: 0.18
- consensus_agreement: 0.12

→ “The verdict was mostly driven by which side had more trustworthy sources.”

Counterfactual Explanations (What Would Have Changed It?)

Generates “what if” scenarios: - “If all sources were high-trust (.gov/.edu), confidence would increase from 65% to ~85%.”

- “If all cited sources were verified, confidence would increase to ~80%.”
- “If arguments contained no fallacies, confidence would increase to ~75%.”

Transparency Score

Scores how “explainable” the decision is (0–1 scale): - +0.3 if sources were cited & checked

- +0.2 if metrics were collected
- +0.2 if fallacies were detected
- +0.3 if calibration metadata exists

Maximum = 1.0 (full transparency). Minimum = 0.0 (no evidence).

Decision Path

A step-by-step log of every decision:

1. CLAIM RECEIVED: “Coffee reduces type 2 diabetes risk”
2. CONSENSUS CHECK: Controversial → proceed to debate
3. EVIDENCE RETRIEVAL: 8 sources
4. MULTI-AGENT DEBATE: 3 rounds
5. SOURCE VERIFICATION: 6 verified
6. ARGUMENTATION ANALYSIS: Fallacies detected (PRO:1, CON:2)
7. CONFIDENCE CALIBRATION: 0.55 → 0.62
8. FINAL VERDICT: PARTIALLY TRUE (62%)

Section 11 : How the AI Providers Are Managed — Circuit Breaker

Code: *src/llm/client.py*

The Problem

Each provider (Groq, Gemini, Cerebras, OpenRouter) has **daily free-tier limits**.

Temporary failures can also occur. You shouldn't keep trying a broken provider.

Solution: Circuit Breaker Pattern

Like your home's electrical panel:

States: - **CLOSED** – everything fine, send requests normally

- **OPEN** – too many failures, stop sending requests

- **HALF_OPEN** – test one request after cooldown

Transition rules: - CLOSED → OPEN after **3 consecutive failures**

- OPEN → HALF_OPEN after **60 seconds**

- HALF_OPEN → CLOSED if test succeeds

- HALF_OPEN → OPEN if test fails (wait another 60s)

Rate Limit Tracking (rolling window)

Provider	Max requests/minute
Groq	28
Gemini	9
Cerebras	28
OpenRouter	18

When a rate limit is hit: 1. Provider gets a **90-second cooldown**

2. System switches to the next provider

API Key Lifecycle

- **ACTIVE** – working
- **RATE_LIMITED** – wait 90s, then retry
- **INVALID** – wrong/expired, skip permanently

Exponential Backoff

For temporary failures (not rate limits): - Retry 1: wait ~2s
- Retry 2: wait ~4s
- Retry 3: wait ~8s
- Max wait: 8s, max retries: 3

Provider Selection Priority

1. Preferred provider (Groq for Pro/FactChecker, Gemini for Con/Moderator)
2. Fallback order: Groq → Gemini → Cerebras → OpenRouter

For each provider, the system checks: - Circuit OPEN? → skip
- Working API keys? → skip
- In cooldown? → skip
- Within rate limit? → skip
- All checks pass → send request

Security: API Key Redaction

Keys are stripped from logs: - `gsk_*` (Groq)
- `AIza*` (Gemini)
- `sk-or-*` (OpenRouter)

SSRF Protection

URLs matching these patterns are blocked: - `localhost`, `127.x.x.x`, `0.0.0.0`
- `10.x.x.x`, `172.16-31.x.x`, `192.168.x.x`
- `169.254.x.x` (cloud metadata)
- IPv6 loopback `::1` or `[::1]`

Section 12 : Putting It All Together — The Full Picture

Here is every algorithm, in order, for a single claim:

Step 1 – Semantic Cache Lookup

- Embed claim $\rightarrow$ 384-number vector
- Cosine similarity vs stored vectors
- If 0.85 $\rightarrow$ return cached answer (<1s), **DONE**

Step 2 – Complexity Estimation

- Score: Semantic(0.3)+Domain(0.3)+Temporal(0.2)+Evidence(0.2)
- Set debate rounds (2–4) and min sources (3–7)

Step 3 – Consensus Pre-Check

- Quick AI check: settled science?
- If confidence > 0.90 $\rightarrow$ return verdict immediately

Step 4 – Adversarial Evidence Retrieval

- Tavily searches for pro/con evidence
- Falls back to Google Search if Tavily limit hit

Step 5 – Debate (N rounds)

- PRO writes argument with URLs
- CON counters with separate URLs
- Both must challenge opponent's previous argument
- After round 2: rolling summary keeps context manageable

Step 6 – Source Verification

- For every cited URL: filter, fetch (10s timeout, 5 workers parallel)
- Embed & cosine similarity vs claim (threshold 0.82)
- Assign trust score by domain
- Label: VERIFIED / NOT_FOUND / CONTENT_MISMATCH / TIMEOUT / PAYWALL / INVALID

Step 7 – Human Review Check

- If both sides' verification rate < 30% $\rightarrow$ agents re-argue once
- Still < 30% $\rightarrow$ LangGraph **pauses** pipeline $\rightarrow$ HITL panel appears
- User reviews sources, can override verdict $\rightarrow$ pipeline resumes from checkpoint

Step 8 – Argumentation Analysis

- Scan for 10 fallacies via pattern matching
- Quality scores per argument, bonuses/penalties applied

Step 9 – Moderator Verdict

$$> \text{composite} = (\text{arg_quality} \times 0.30) + (\text{avg_verification} \times 0.30) + (\text{avg_trust} \times 0.20) + (\text{consensus} \times 0.20)$$

Step 10 – Contradiction Detection

- Compare VERIFIED sources pairwise

- Detect directional, temporal, jurisdictional contradictions
- Generate verdict modifiers if needed

Step 11 – Confidence Calibration

- Calculate source_quality (geometric mean) & debate_asymmetry
- Multi-factor under-confidence detection
- Apply boost (max +0.25), cap at 0.95

Step 12 – Explainability

- Compute feature importance (normalised to 1.0)
- Generate counterfactual explanations
- Build decision path & transparency score

Step 13 – Cache Write

- Store embedding + verdict JSON in SQLite (TTL 7 days)
- Next similar claim returns instantly

Step 14 – Stream to User

- All results sent via Server-Sent Events: verdict, confidence, transcript, source table, feature importance, fallacy flags, calibration details, decision path.

Key Numbers Summary (from the actual code)

Parameter	Value
Cache similarity threshold	0.85
Verification threshold	0.82
HITL trigger	< 30% verification rate
Calibration max boost	0.25
Calibration cap	0.95 (never 100%)
Geometric mean floor	0.01 (prevents log(0) crash)
Moderator weights	30%+30%+20%+20% = 100%
Circuit breaker threshold	3 consecutive failures → OPEN
Cooldown after rate limit	90 seconds
URL fetch timeout	10 seconds
Parallel URL workers	5
Cache TTL	7 days
Max provider cooldown	8 seconds
API key redaction patterns	gsk_, AIza, sk-or-*

END OF MATH AND ALGORITHMS GUIDE

If you want to go deeper, the actual code files are:

- Formulas: `src/agents/moderator.py`
- Cache: `src/orchestration/cache.py`
- Complexity: `src/novelty/claim_complexity.py`
- Trust scoring: `src/utils/trust_scorer.py`
- Verification: `src/agents/fact_checker.py`
- Calibration: `src/novelty/confidence_calibration.py`
- Fallacies: `src/novelty/argumentation_analysis.py`
- Contradictions: `src/novelty/contradiction_detection.py`
- Explainability: `src/novelty/explainability.py`
- API resilience: `src/llm/client.py`
- Search: `src/utils/tavily_retriever.py`